

Claude Code Skill — Frontmatter & Body Reference

Source: <https://code.claude.com/docs/en/skills>

Structure

A skill is a directory with a required `SKILL.md`, plus optional supporting files:

```
my-skill/
├─ SKILL.md           # required – overview and instructions
├─ reference.md       # optional – loaded only when needed
├─ examples.md        # optional – usage examples
└─ scripts/
   └─ helper.py       # executed, not loaded into context
```

Legacy `.claude/commands/deploy.md` files still work — same frontmatter fields apply, just no directory or supporting files.

Frontmatter Reference

All fields are optional. Only `description` is recommended, since it's what Claude uses to decide when to load the skill.

Field	Required	Description
<code>name</code>	No	Display name shown in skill listings. Defaults to the directory name.
<code>description</code>	Recommended	What the skill does and when to use it. If omitted, uses the first paragraph of markdown content. Put the key use case first — combined with <code>when_to_use</code> , it's truncated at 1,536 characters.
<code>when_to_use</code>	No	Additional trigger phrases or example requests, appended to <code>description</code> . Counts toward the same 1,536-char cap.
<code>argument-hint</code>	No	Autocomplete hint for expected arguments, e.g. <code>[issue-number]</code> or <code>[filename] [format]</code> .
<code>arguments</code>	No	Named positional arguments for <code>\$name</code> substitution. Accepts a space-separated string or YAML list.
<code>disable-model-invocation</code>	No	<code>true</code> = only you can invoke it via <code>/name</code> ; Claude won't auto-trigger it. Use for side-effecting actions like <code>/commit</code> , <code>/deploy</code> . Also blocks preloading into subagents and scheduled-task firing. Default: <code>false</code> .
<code>user-invocable</code>	No	<code>false</code> = hidden from the <code>/</code> menu; only Claude can invoke it. Use for background knowledge, not actionable commands. Default: <code>true</code> .
<code>allowed-tools</code>	No	Tools Claude can use without asking permission during the invoking turn. Clears on your next message. Space/comma-separated string or YAML list.

disallowed-tools	No	Tools removed from Claude's pool while the skill is active. Clears on your next message. Can't remove <code>EndConversation</code> while any other tool remains.
model	No	Model override while this skill is active, for the rest of the turn only. Accepts same values as <code>/model</code> , or <code>inherit</code> .
effort	No	Effort level while active: <code>low</code> , <code>medium</code> , <code>high</code> , <code>xhigh</code> , <code>max</code> . Default: inherits from session.
context	No	Set to <code>fork</code> to run in an isolated forked subagent context (no access to conversation history).
agent	No	Which subagent type to use when <code>context: fork</code> is set (<code>Explore</code> , <code>Plan</code> , <code>general-purpose</code> , or <code>custom</code>). Defaults to <code>general-purpose</code> .
hooks	No	Hooks scoped to this skill's lifecycle.
paths	No	Glob patterns limiting when the skill auto-activates — only loads automatically when working with matching files. Comma-separated string or YAML list.
shell	No	Shell for <code>!`command`</code> injection: <code>bash</code> (default) or <code>powershell</code> .

Invocation control cheat sheet

Frontmatter	You can invoke	Claude can invoke	Context loading
<i>(default)</i>	Yes	Yes	Description always in context; full content loads on invoke
<code>disable-model-invocation: true</code>	Yes	No	Description not in context; full content loads only when you invoke
<code>user-invocable: false</code>	No	Yes	Description always in context; full content loads on invoke

Example frontmatter block

```
---
name: deploy
description: Deploy the application to production
disable-model-invocation: true
allowed-tools: Bash(npm run build) Bash(npm test)
context: fork
agent: general-purpose
---
```

Body

The markdown content after the frontmatter is the skill's instructions. It can contain:

1. Plain instructions

Step-by-step procedures, checklists, conventions, or reference knowledge — whatever you'd otherwise paste into chat repeatedly.

2. Argument placeholders

Placeholder	Meaning
<code>\$ARGUMENTS</code>	All arguments passed, as a single string
<code>\$ARGUMENTS[N]</code>	Specific argument by 0-based index
<code>\$N</code>	Shorthand for <code>\$ARGUMENTS[N]</code> (e.g. <code>\$0</code> , <code>\$1</code>)
<code>\$name</code>	Named argument from the <code>arguments</code> frontmatter list

Escape a literal `$` before a digit/name with a backslash: `\$1.00` .

3. Built-in variables

Variable	Value
<code>\${CLAUDE_SESSION_ID}</code>	Current session ID
<code>\${CLAUDE_EFFORT}</code>	Active effort level
<code>\${CLAUDE_SKILL_DIR}</code>	Directory containing this skill's <code>SKILL.md</code> — use for referencing bundled scripts
<code>\${CLAUDE_PROJECT_DIR}</code>	Project root directory

4. Dynamic context injection

`!`command`` runs a shell command *before* Claude sees the prompt; the output replaces the placeholder inline.

```
## Current changes
!`git diff HEAD`
```

For multi-line commands, use a fenced block:

```
```!
node --version
npm --version
```
```

Only recognized when `!` starts a line or follows whitespace — `KEY=! cmd`` won't trigger it.

5. References to supporting files

```
## Additional resources
- For complete API details, see [reference.md](reference.md)
- For usage examples, see [examples.md](examples.md)
```

Keep `SKILL.md` itself under ~500 lines; push detailed reference material into separate files that load only when needed.

Two content patterns

Reference content (runs inline, informs current work): ``yaml

name: api-conventions description: API design patterns for this codebase

When writing API endpoints:

- Use RESTful naming conventions
- Return consistent error formats
- Include request validation

```
**Task content** (explicit action, usually manually triggered):
```

```
``yaml
```

```
---
```

```
name: deploy
```

```
description: Deploy the application to production
```

```
context: fork
```

```
disable-model-invocation: true
```

```
---
```

```
Deploy the application:
```

- ```
1. Run the test suite
2. Build the application
3. Push to the deployment target
```